RTO Power Panel: GCP Deployment Plan

Deploy as a Cloud Run service through the existing analytics-scripts repo (SupertailsPCPL/analytics-scripts) | Prepared 3 Oct 2026

1. Goal and approach

Run the RTO Power Panel (Flask app) on GCP in **sup-internal-apps / asia-south1** as a private Cloud Run service, and ship every future change through a GitHub pull request. We follow the shared repo's conventions rather than build a separate pipeline: one shared Docker image, workloads declared in `deploy/*.conf`, secrets in Secret Manager, and Cloud Build deploying on merge.

Decision: deploy as a Service (gunicorn + Flask), with a scheduled Job refreshing the data caches. This fits the repo's own "Service + Job" pattern.

2. What has to change in the app

Area	Today	Change needed
Layout	Flat files at repo root; app.run(127.0.0.1:5659, debug=True)	Move to `services/rto_panel/` with `__init__.py` and `main.py` exposing `app`; use package imports; no debug; serve with gunicorn on 0.0.0.0:8080.
Entries store	SQLite file (rto_entries.db)	Move to a shared MySQL table (Cloud Run instances have no persistent disk and up to 10 can run at once). The shared DB user may be read-only, so owners must say which schema allows writes.
Caches (about 90 MB)	CSV/JSON files refreshed by background threads	Scheduled Cloud Run **Job** refreshes them into GCS (or MySQL); the service only reads. Threads stall on Cloud Run because CPU is paused between requests.
Google Sheets/Drive	service_account.json key file	Remove the key. Use the runtime service account via Application Default Credentials. Share the Sheet and both Drive folders with that account as **Editor**. Make the OAuth fallback import in google_services.py lazy.
Secrets	.env (SCM_DB_*, DN_DB_*, GOOGLE_SHEET_ID, DRIVE_*)	**DB secrets already exist** in `deploy/access.conf`. SCM DB = `analytics-scripts-mysql-user/-password/-host` (same server and `scm` schema that jobs/scm/control_tower.py reads). Delivery Note ERP = `analytics-scripts-erp-read-replica-user/-password` (host 10.190.0.9, same as the repo's ERP jobs). Add one access.conf line mapping them to the app's SCM_DB_* / DN_DB_* names, so there are no new DB secrets. Port, DB names and the ERP host become constants in code, per repo convention. Only the Sheet and Drive IDs are new.
Uploads	Up to 500 MB through Flask	Cloud Run limits HTTP/1 requests to 32 MiB (default timeout 300 s). Browser uploads straight to GCS with a signed URL; the server then copies the file to Drive.
Timezone	Bare datetime.now() in code	Containers run in UTC. Use explicit IST helpers (see the repo's now_ist()).
Access	No login in app.py	Private service. Owners add Identity-Aware Proxy so staff sign in with company Google accounts.

3. Step-by-step plan

#	Step	Who	Output
0	Send open questions (section 6) to repo owners; confirm which branch deploys (README says main, link points at production).	You	Answers on branch, entries DB, IAP, memory

#	Step	Who	Output
1	Fix .gitignore (add rto_mdnd_*cache.json, rto_recon_cache.json, recon_dn_cache.json, *.tmp); keep .env and service_account.json out of git.	Dev	Safe to commit
2	Create a working branch in analytics-scripts; refactor into services/rto_panel/ (layout, imports, gunicorn entry, health endpoint).	Dev	App imports and runs from the new path
3	Replace SQLite with the shared MySQL table; migrate existing rows once.	Dev + owner	Entries persisted off-instance
4	Add the cache-refresh Job (jobs/rto_panel/refresh_caches.py) and a cron line in schedules.conf; service reads from GCS.	Dev	No background threads in the service
5	Add signed-URL upload flow; switch Drive/Sheets calls to Application Default Credentials; fix timezones.	Dev	Large uploads work; no key file
6	Add deploy lines: services.conf (rto-panel = gunicorn services.rto_panel.main:app --bind=0.0.0.0:8080), jobs.conf, schedules.conf, access.conf (reusing the existing DB secrets).	Dev + owner	Workload declared
7	Owners: review the new access.conf line, add secrets for the Sheet/Drive IDs only, run sync_access.sh, set up IAP, raise memory/timeout if needed.	Owners	Access line, IAP in place
8	Share Sheet and Drive folders with the runtime account (Editor). Test locally with the repo's docker build/run commands.	You	Local container test passes
9	Open PR. CODEOWNERS reviews build-file changes. Merge triggers Cloud Build, which prints the service URL.	Dev + owner	Service live (private)
10	Smoke test via IAP: create one entry, check the Sheet row, Drive upload, dropdowns, RTO Delivered / MDND / Recon tabs. Run the refresh job manually once.	You	Signed-off checklist
11	Announce to users; keep the local app as fallback for one week.	You	Cutover done

4. How future changes ship

- Branch from main, edit `services/rto_panel/**`, run the app locally in Docker (`docker build -t analytics-scripts .` then run with the module args).
- Open a PR. Changes to services/, jobs/ and non-access deploy files can be reviewed by anyone; changes to Dockerfile, cloudbuild.yaml, requirements.txt and access.conf need an owner.
- Merge to the deploying branch. Cloud Build builds the image and redeploys every workload declared in deploy/, so test thoroughly first: a merge touches the shared image, not only this app.

5. Risks and rollback

Risk	Mitigation
A merge redeploys all jobs and services (shared blast radius)	Keep PRs small; test in the local container; avoid touching shared build files.
No staging or PR preview environment in the repo	Ask owners whether a staging service is possible; otherwise test locally and smoke test right after deploy.
Writes to the real Sheet and Drive during tests	Use a test Sheet and test Drive folders in local runs (separate env values).
Service OOM at default memory with large caches	Request more memory; read caches lazily instead of loading everything at start.

Risk	Mitigation
Shared DB users lack grants the app needs	Network is fine: existing jobs reach both hosts over prod-vpc. Confirm the shared SCM user can SELECT scm.cp_database_N and the ERP replica user can SELECT the Delivery Note tables; the app only reads these.
Bad release	Revert the PR and merge (redeploys the previous code), or shift Cloud Run traffic to the previous revision.

6. Open questions for the repo owners

- Which branch triggers deploys: main or production?
- Can a table in an existing MySQL hold the entries? The shared DB user may be read-only, so which schema allows writes?
- Can IAP be enabled for rto-panel, and the Sheet and Drive IDs added as secrets (the DB secrets already exist and will be reused)?
- Do the shared SCM MySQL user and the ERP read-replica user have SELECT on scm.cp_database_N and the Delivery Note tables the app queries?
- Can services get more memory, a longer timeout or always-on CPU if needed?
- Is there any PR check or staging service I did not see? (.github/workflows returned 404 when I read the repo.)

7. Not verified

The analytics-scripts repo was read through the GitHub API only: I did not read CODEOWNERS or resources.conf, and I did not see the Cloud Build trigger configuration. I could not read secret values, so that the shared DB secrets point at the same servers as the local .env is inferred from the hosts the repo's jobs use (analytics-db.supertails.com / scm and 10.190.0.9), not confirmed. Memory, timeout and branch behaviour for services should be confirmed with the owners before relying on this plan.